Code Smell, Anti-pattern, dan Refactoring

Mata Kuliah:
CIF61123 – Pola Perancangan (3 SKS)
Pekan ke-6 | Sub-CPMK13
Disusun oleh:
Kelompok Pengelola Mata Kuliah Pola Perancangan



Versi Dokumen: 1.0
Tanggal: 17; Bulan: Agustus; Tahun: 2026
© Fakultas Ilmu Komputer, Universitas Brawijaya

filkom.ub.ac.id

Peta Perkuliahan

Kedudukan pertemuan ini di antara enam belas pekan perkuliahan

01	Konsep dasar & klasifikasi pola
02	Prinsip perancangan berorientasi objek
03	Interface, coupling–cohesion, UML
04	SOLID bagian 1 — SRP, OCP, LSP
05	SOLID bagian 2 — ISP, DIP
06	**Code smell & refactoring**
07	Klaster 1 — kontrol pembuatan objek
08	Ujian Tengah Semester
09	Klaster 2 — komposisi rekursif
10	Klaster 3 — pembungkus antarmuka
11	Klaster 4 — algoritma bervariasi
12	Klaster 5 — ketergantungan longgar
13	Katalog GoF & pedoman pemilihan
14	Studi kasus terpadu bagian 1
15	Studi kasus terpadu bagian 2
16	Ujian Akhir Semester

© Fakultas Ilmu Komputer, Universitas Brawijaya

SubCPMK yang Didukung

Pertemuan penutup Sub-CPMK13 dan jembatan menuju pola perancangan

13 Sub-CPMK13

Mampu menganalisis penerapan prinsip SOLID dan melakukan refactoring pada kode program Java.

Bobot pada RPS: 15,00% dari nilai akhir.

S Bentuk penilaian pekan ini

Sumatif — tugas analisis dan refactoring (LK02).

Bobot: 4,00% melalui TUGAS2.

Diumumkan pekan ini, dikumpulkan akhir pekan ke-7.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Materi Pembelajaran

Empat pokok bahasan, bersumber dari [2], P3, dan P4

1 Code smell

Tanda pada kode yang menunjukkan rancangan bermasalah.

2 Anti-pattern

Solusi yang tampak wajar tetapi berakibat buruk.

3 Teknik refactoring

Perubahan struktur tanpa mengubah perilaku program.

4 Penjelasan Tugas 2

Lingkup, luaran, kriteria penilaian, dan jadwal LK02.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Tujuan Pembelajaran

Setelah pertemuan ini mahasiswa diharapkan mampu:

- Mengidentifikasi code smell pada kode program Java dan menjelaskan mengapa hal itu menjadi masalah.
- Mengenali anti-pattern yang menyertai code smell tersebut.
- Menjelaskan disiplin refactoring dan peran pengujian sebagai jaring pengaman.
- Menerapkan teknik refactoring yang tepat untuk memperbaiki pelanggaran prinsip SOLID.
- Melakukan refactoring bertahap tanpa mengubah perilaku program yang dapat diamati.

© Fakultas Ilmu Komputer, Universitas Brawijaya

01

MATERI KE-1

Code Smell

Bagian 1 dari 4

© Fakultas Ilmu Komputer, Universitas Brawijaya

Apa Itu Code Smell

"A code smell is a surface indication that usually corresponds to a deeper problem in the system." — Fowler

Pengertian

- **Istilah Kent Beck dan Martin Fowler.** Ciri pada kode yang menandakan kemungkinan adanya masalah rancangan yang lebih dalam.
- **Bukan kesalahan.** Program tetap berjalan benar; yang terganggu adalah kemudahan perubahannya.
- **Bukan vonis.** Smell adalah petunjuk untuk diperiksa, bukan aturan yang wajib dituruti.

Mengapa dipelajari

- **Bahasa yang tepat.** Memberi nama pada masalah membuat diskusi rancangan menjadi terarah.
- **Titik masuk perbaikan.** Setiap smell memiliki teknik refactoring yang sudah baku.
- **Jembatan menuju pola.** Sebagian besar pola GoF adalah tujuan akhir dari refactoring atas smell tertentu.

◆ **Intinya: smell menunjukkan letak masalah; prinsip SOLID menjelaskan mengapa itu masalah.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Katalog Code Smell yang Wajib Dikenali

Delapan smell yang paling sering muncul pada tugas dan ujian

Nama	Ciri	Prinsip yang biasanya dilanggar
Long Method	Satu metode terlalu panjang dan mengerjakan banyak hal	SRP
Large Class / God Object	Satu kelas mengurusi hampir seluruh sistem	SRP
Feature Envy	Metode lebih banyak memakai data kelas lain daripada data kelasnya sendiri	SRP, enkapsulasi
Switch Statements	Percabangan panjang atas jenis objek yang berulang di banyak tempat	OCP
Refused Bequest	Turunan menolak sebagian warisan induknya	LSP
Shotgun Surgery	Satu perubahan kebutuhan menuntut banyak kelas ikut diubah	SRP, coupling
Divergent Change	Satu kelas berubah karena beberapa alasan yang berbeda	SRP
Data Class	Kelas hanya berisi data tanpa perilaku apa pun	Enkapsulasi

© Fakultas Ilmu Komputer, Universitas Brawijaya

Contoh: Long Method dan Feature Envy

Satu metode yang menghitung, menyusun teks, sekaligus mencetak

LaporanNilai.java

```
class LaporanNilai {
    void cetak(Mahasiswa mhs, List<Krs> daftar) {
        double totalBobot = 0, totalSks = 0;
        for (Krs k : daftar) {                          // Feature Envy:
            totalBobot += k.nilai().angka()             // memakai data Krs
                        * k.mataKuliah().sks();         // dan MataKuliah,
            totalSks   += k.mataKuliah().sks();         // bukan data
sendiri
        }
        double ipk = totalBobot / totalSks;
        StringBuilder sb = new StringBuilder();         // Long Method:
        sb.append("LAPORAN NILAI\n");                   // menghitung,
        sb.append(mhs.nama()).append("\n");             // menyusun teks,
        for (Krs k : daftar) {
            sb.append(k.mataKuliah().nama()).append("\n");
        }
        System.out.println(sb);                         // dan mencetak
    }
}
```

Perhitungan IPK seharusnya menjadi milik kelas yang memiliki datanya.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Cara Menemukan Smell tanpa Menjalankan Program

Empat pemeriksaan cepat yang dapat dipakai pada LK02

1 Ukur panjangnya

Metode lebih dari dua puluh baris dan kelas lebih dari tiga ratus baris layak diperiksa.

2 Hitung titiknya

Rangkaian a.b().c().d() menandakan kelas ini tahu terlalu banyak tentang tetangganya.

3 Cari percabangan berulang

Switch atas jenis objek yang sama muncul di lebih dari satu berkas.

4 Telusuri satu perubahan

Bayangkan satu permintaan perubahan, lalu hitung berapa berkas yang tersentuh.

© Fakultas Ilmu Komputer, Universitas Brawijaya

02

MATERI KE-2

Anti-pattern

Bagian 2 dari 4

© Fakultas Ilmu Komputer, Universitas Brawijaya

Membedakan Code Smell dan Anti-pattern

Anti-pattern adalah kebalikan pola perancangan: sama-sama berulang, tetapi akibatnya merugikan

Code smell

- **Cakupan.** Ciri lokal pada satu metode atau satu kelas.
- **Sifat.** Petunjuk bahwa ada sesuatu yang perlu diperiksa.
- **Contoh.** Long Method, Feature Envy, Data Class.
- **Penanganan.** Satu atau dua teknik refactoring biasanya cukup.

Anti-pattern

- **Cakupan.** Pola penyelesaian pada tingkat rancangan atau arsitektur.
- **Sifat.** Solusi yang tampak masuk akal tetapi berakibat buruk dan berulang.
- **Contoh.** God Object, Spaghetti Code, Golden Hammer, Lava Flow.
- **Penanganan.** Menuntut penataan ulang rancangan, bukan sekadar merapikan kode.

Intinya: setiap anti-pattern memiliki solusi baku yang sudah dikenali — sebagaimana setiap pola memiliki masalah bakunya.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Anti-pattern yang Sering Ditemui

Enam anti-pattern beserta arah penyelesaiannya

1 God Object

Satu kelas mengetahui dan mengatur hampir segalanya.

Solusi: pisahkan menurut aktor, terapkan SRP.

2 Spaghetti Code

Alur kendali berbelit tanpa struktur yang jelas.

Solusi: ekstrak metode, ganti percabangan dengan polimorfisme.

3 Golden Hammer

Satu teknik dipakai untuk segala persoalan.

Solusi: kuasai katalog pola agar pilihan bertambah.

4 Lava Flow

Kode mati yang dibiarkan karena tidak ada yang berani menghapusnya.

Solusi: pengujian yang memadai, lalu hapus.

5 Poltergeist

Kelas yang hanya meneruskan panggilan tanpa tanggung jawab nyata.

Solusi: hapus dan hubungkan langsung.

6 Singleton yang berlebihan

Keadaan global disamarkan sebagai pola.

Solusi: dibahas tuntas pada pekan ke-7.

© Fakultas Ilmu Komputer, Universitas Brawijaya

03

MATERI KE-3

Refactoring

Bagian 3 dari 4

© Fakultas Ilmu Komputer, Universitas Brawijaya

Refactoring adalah Disiplin, Bukan Sekadar Merapikan

Perbedaan ini menentukan cara Anda menyusun riwayat commit pada LK02

Definisi Fowler

- “A change made to the internal structure of software to make it easier to understand and cheaper to modify without changing its observable behaviour.”
- **Kata kuncinya.** Tanpa mengubah perilaku yang dapat diamati.
- **Ukurannya.** Seluruh kasus uji yang semula lolos harus tetap lolos.

Yang bukan refactoring

- **Menambah fitur.** Menambah perilaku baru sambil menata ulang struktur.
- **Memperbaiki kesalahan.** Mengubah perilaku yang keliru menjadi benar.
- **Menulis ulang.** Membuang kode lama dan menggantinya dengan yang baru.
- **Alasannya.** Ketiganya sah dilakukan, tetapi tidak boleh dicampur dengan langkah refactoring.

Intinya: satu commit hanya boleh berisi satu jenis pekerjaan — menata ulang, atau mengubah perilaku.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Pengujian sebagai Jaring Pengaman

Tanpa pengujian, yang Anda lakukan bukan refactoring melainkan menebak

1 Jalankan dahulu

Pastikan seluruh kasus uji yang disertakan lolos sebelum satu baris pun diubah.

Inilah basis perilaku awal.

2 Ubah sedikit demi sedikit

Satu teknik refactoring, satu langkah.

Jangan menumpuk beberapa perubahan sekaligus.

3 Uji ulang tiap langkah

Jalankan kembali seluruh kasus uji setelah setiap langkah selesai.

Gagal berarti langkah itu dibatalkan, bukan ditambal.

4 Simpan tiap langkah

Satu langkah refactoring, satu commit dengan pesan yang menjelaskan tekniknya.

Riwayat inilah yang dinilai pada LK02.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Katalog Teknik Refactoring yang Dipakai di Mata Kuliah Ini

Kolom terakhir menunjukkan bahwa refactoring bermuara pada prinsip dan pola

Teknik	Dipakai ketika	Mengarah pada
Extract Method	Satu metode mengerjakan lebih dari satu hal	SRP pada tingkat metode
Extract Class	Satu kelas melayani lebih dari satu aktor	SRP pada tingkat kelas
Move Method	Metode lebih banyak memakai data kelas lain	Menghilangkan Feature Envy
Extract Interface	Klien hanya memerlukan sebagian operasi	ISP dan DIP
Replace Conditional with Polymorphism	Percabangan atas jenis objek berulang	OCP, pola Strategy dan State
Replace Constructor with Factory Method	Pemilihan jenis objek tersebar di kode klien	DIP, pola Factory Method
Introduce Parameter Object	Sekelompok parameter selalu muncul bersama	Mengurangi stamp coupling

© Fakultas Ilmu Komputer, Universitas Brawijaya

Teknik 1 — Extract Class

Aturan tentang alamat kini menjadi milik kelas yang memiliki datanya

Sebelu

```
class Mahasiswa {
    private String nim, nama;
    private String jalan, kota, kodePos;

    String alamatLengkap() {
        return jalan + ", " + kota
             + " " + kodePos;
    }
    boolean kodePosValid() {
        return kodePos.matches("\\d{5}");
    }
}
```

Sesuda

```
class Alamat {
    private final String jalan, kota, kodePos;

    String lengkap() {
        return jalan + ", " + kota + " " + kodePos;
    }
    boolean kodePosValid() {
        return kodePos.matches("\\d{5}");
    }
}

class Mahasiswa {
    private String nim, nama;
    private Alamat alamat;
}
```

- **Perubahan aturan penulisan kode pos tidak lagi menyentuh kelas Mahasiswa.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Teknik 2 — Replace Conditional with Polymorphism

Jenis pinjaman baru cukup menambah satu kelas tarif

Sebelu

```
class Denda {
    double hitung(Pinjaman p) {
        switch (p.jenis()) {
            case "BUKU":
                return p.hariTerlambat() * 1000;
            case "SKRIPSI":
                return p.hariTerlambat() * 2500;
            case "REFERENSI":
                return p.hariTerlambat() * 5000;
            default:
                return 0;
        }
    }
}
```

Sesuda

```
interface TarifDenda {
    double perHari();
}

class TarifBuku implements TarifDenda {
    public double perHari() { return 1000; }
}

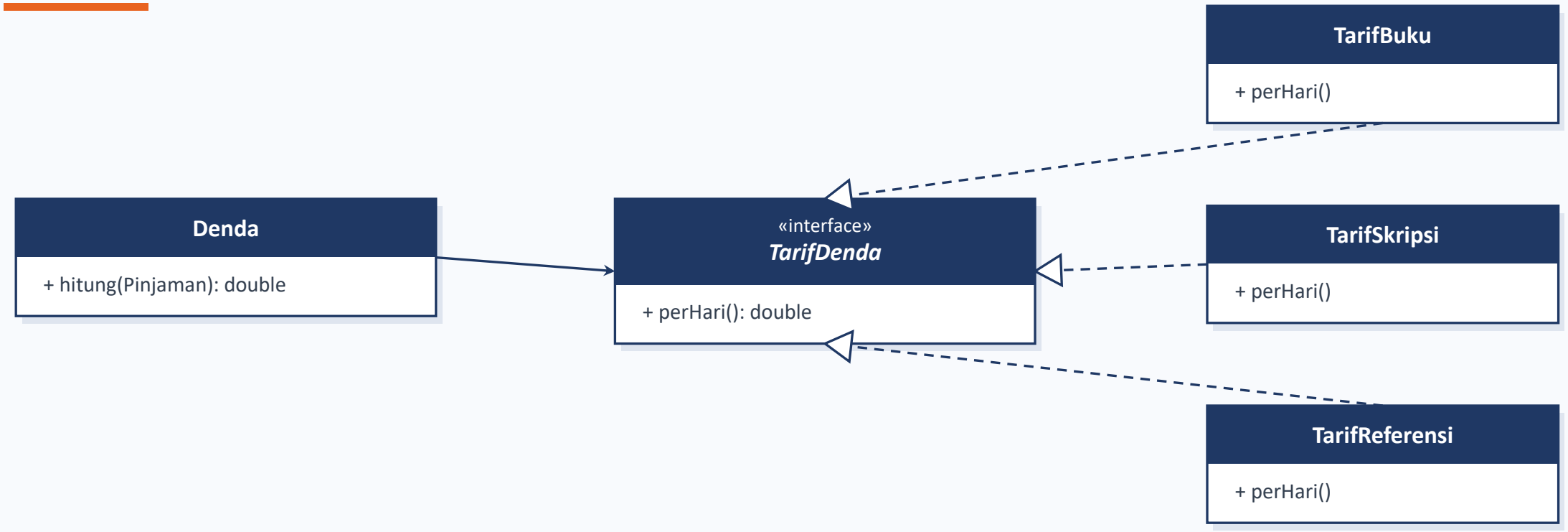
class Denda {
    double hitung(Pinjaman p) {
        return p.hariTerlambat()
             * p.tarif().perHari();
    }
}
```

- **Struktur hasil refactoring ini persis pola Strategy, yang dibahas pada pekan ke-11.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Rancangan Setelah Kedua Teknik Diterapkan

Bandingkan dengan struktur OCP pada pekan ke-4 — keduanya identik



- **Refactoring yang disiplin menuntun rancangan menuju pola tanpa perlu menghafal polanya.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Refactoring Menuju Pola Perancangan

Enam smell dan pola yang menjawabnya — peta jalan paruh kedua semester

- **Percabangan atas jenis objek** → Strategy atau State (pekan ke-11).
- **Kata kunci new yang tersebar** → Factory Method atau Abstract Factory (pekan ke-7).
- **Ledakan subkelas karena kombinasi kemampuan** → Decorator (pekan ke-9).
- **Antarmuka pustaka pihak ketiga yang tidak sesuai** → Adapter (pekan ke-10).
- **Modul yang harus diberi tahu setiap kali keadaan berubah** → Observer (pekan ke-12).
- **Struktur berjenjang yang diperlakukan berbeda dari daunnya** → Composite (pekan ke-9).

◆ **Intinya: mulai pekan ke-7 kita tidak lagi menemukan pola, melainkan mengenali kembali apa yang sudah kita lakukan.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

04

MATERI KE-4

Tugas 2 — Analisis SOLID dan Refactoring

Bagian 4 dari 4

© Fakultas Ilmu Komputer, Universitas Brawijaya

LK02 — Lingkup Pekerjaan

Tugas individu, mengukur Sub-CPMK13 dengan bobot 4,00%

1 Masukan

Berkas kode program Java yang berfungsi benar namun mengandung pelanggaran prinsip SOLID dan code smell.

Disertai kasus uji yang seluruhnya lolos.

2 Yang dikerjakan

Telusuri kode, catat pelanggaran beserta lokasinya.

Kelompokkan temuan menurut prinsip yang dilanggar.

Refactoring bertahap dengan pengujian ulang tiap tahap.

3 Luaran

Laporan PDF berisi tabel temuan, potongan kode sebelum dan sesudah, serta class diagram akhir.

Repositori Git dengan riwayat commit yang menunjukkan tahapan perbaikan.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Kriteria dan Bobot Penilaian LK02

Diambil langsung dari berkas Lembar Kerja

Kriteria	Bobot	Yang dinilai
Ketepatan identifikasi pelanggaran	30%	Kelengkapan temuan dan kebenaran pemetaan temuan ke prinsip SOLID yang dilanggar
Kualitas hasil refactoring	35%	Rancangan akhir memenuhi prinsip SOLID dan tidak menimbulkan pelanggaran baru
Kesetaraan perilaku program	20%	Seluruh kasus uji tetap lolos setelah refactoring dilakukan
Kelengkapan dan kejelasan laporan	15%	Keterbacaan tabel temuan, kejelasan penjelasan, dan konsistensi diagram akhir

© Fakultas Ilmu Komputer, Universitas Brawijaya

Jadwal Pelaksanaan LK02

Pengumpulan paling lambat pukul 23.59 pada hari terakhir

Waktu	Aktivitas	Luaran
Pekan ke-6	Penjelasan tugas dan pembagian berkas kode program	Hasil eksekusi kasus uji awal
Pekan ke-6 s.d. ke-7	Analisis pelanggaran dan pelaksanaan refactoring	Tabel temuan dan kode hasil perbaikan
Akhir pekan ke-7	Pengumpulan luaran melalui LMS BRONE	Laporan dan tautan repositori

Perhatikan bahwa KUIS1 juga dilaksanakan pada awal pekan ke-7, mencakup Sub-CPMK11, Sub-CPMK12, dan Sub-CPMK13.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Format Tabel Temuan yang Diminta

Empat kolom — gunakan format ini agar penilaian berjalan cepat dan adil

1 Lokasi

Nama kelas dan nomor baris tempat pelanggaran ditemukan.

2 Temuan

Nama code smell dan prinsip SOLID yang dilanggar.

3 Alasan

Mengapa potongan kode itu melanggar prinsip tersebut.

4 Perbaikan

Teknik refactoring yang dipakai beserta hasilnya.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Simpulan

- Code smell adalah ciri pada kode yang menandakan kemungkinan masalah rancangan yang lebih dalam, bukan kesalahan program.
- Anti-pattern adalah solusi berulang pada tingkat rancangan yang tampak masuk akal tetapi berakibat merugikan.
- Refactoring adalah perubahan struktur tanpa mengubah perilaku yang dapat diamati; pengujian adalah jaring pengamannya.
- Setiap teknik refactoring mengarah pada pemenuhan prinsip SOLID tertentu, dan sebagian besar bermuara pada pola perancangan.
- Tugas 2 (LK02) menuntut tabel temuan, refactoring bertahap yang terekam pada riwayat commit, dan kasus uji yang tetap lolos.

Pekan depan: klaster pertama pola perancangan — kontrol pembuatan objek: Factory Method, Singleton, dan Abstract Factory. KUIS1 juga dilaksanakan pada awal pertemuan.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Pustaka

Gaya pengacuan: Harvard-Anglia

- Fowler, M., 2018. Refactoring: Improving the Design of Existing Code. Edisi ke-2. Boston, MA: Addison-Wesley.
- Martin, R.C., 2002. Agile Software Development: Principles, Patterns, and Practices. Upper Saddle River, NJ: Prentice Hall.
- Refactoring Guru, 2024. Refactoring dan Design Patterns. [daring] Tersedia pada: <https://refactoring.guru> [Diakses 5 Juli 2024].
- Kerievsky, J., 2004. Refactoring to Patterns. Boston, MA: Addison-Wesley.

© Fakultas Ilmu Komputer, Universitas Brawijaya